

INITIATION À L'ENVIRONNEMENT UNIX : TP5
novembre 2024 — Pierre Rousselin

1 Processus

Exercice 1 : Les commandes `ps` et `pstree`

1. Si elle est installée sur votre système, lancez la commande `pstree -T -p`. Que fait-elle ?
 - a) Quel processus est à la racine de cette arborescence ?
 - b) Si ce n'est pas déjà fait, ouvrez un navigateur web (et dans ce cas relancez la commande `pstree -T -p`). Combien de processus enfants a-t-il ?
2. À l'aide d'une commande `ps` bien choisie (voir les transparents du cours ou le manuel de `ps`) :
 - a) Afficher des informations sur le ou les processus de nom de base `systemd` et uniquement celui-là ou ceux-là. Quel est leur utilisateur propriétaire ?
 - b) Afficher des informations sur les processus dont vous êtes propriétaires et seulement ceux-là.
 - c) Donner l'ordre de grandeur du nombre total de processus vivant à cet instant sur votre machine à l'aide de la commande `ps -e | wc -l`. Parmi ceux-ci, combien appartiennent à `root` ?

--- * ---

Exercice 2 : À la recherche de nos ancêtres

1. Afficher le PID du shell courant.
2. Donner deux façons d'afficher le PPID (c'est-à-dire le PID du parent) du shell courant, l'une avec `ps`, l'autre avec un développement de variables.
3. À l'aide de `ps` trouver à quel processus (nom de commande) correspond le parent du shell courant. Chercher le PPID de ce processus.
4. Continuer ainsi jusqu'à arriver jusqu'au processus de PID 1, de sorte à pouvoir décrire tous les ancêtres du shell courant.

--- * ---

Exercice 3 : Pour rigoler

1. Recopier dans un fichier `lol` le script suivant :

```
#!/bin/sh
# lol : rigoler après avoir dormi 10 secondes
sleep 10; echo lol
```

Le rendre exécutable et l'essayer.

2. Recopier dans un fichier `deuxfois` le script suivant :

```
#!/bin/sh
# deuxfois : exécuter deux fois une commande
"$@" & "$@"; wait
```

Ce script exécute deux fois la commande formée par ses arguments, se faisant, *fork* deux fois et attend que ses enfants se terminent avec la primitive `wait`. Vous n'avez pas besoin de le comprendre entièrement maintenant.

Le rendre exécutable et tester la commande `./deuxfois ls /`.

3. Entrer la suite de commandes suivantes :

```
$ pstree -p -T $$
$ ./deuxfois ./deuxfois ./lol &
$ pstree -p -T $$
```

Combien de processus ont été créés par la deuxième de ces commandes ?

- Essayer de prévoir le nombre de `lol` affichés et le nombre de processus créés par la commande
`$ ./deuxfois ./deuxfois ./deuxfois ./deuxfois ./deuxfois ./lol &`
 Vérifier en visualisant avec la commande `pstree` ci-dessus.

--- * ---

2 Arguments de la ligne de commande

Exercice 4 : Une petite calculatrice

- Recopier le script suivant, appelé `calcul`

```
#!/bin/sh
printf "Nombre d'arguments : $#\n"
printf '%d\n' $(( $1 + $2 ))
```
- Rendre ce script exécutable, puis entrer les commandes

```
$ ./calcul 13 8
$ ./calcul 6 8 12 1
$ ./calcul 89
```
- À quoi correspondent `$1`, `$2` et `$#` ?
- Modifier ce script, de façon à ce que s'il y a 0 ou 1 argument, le script affiche un message d'erreur. Enlever l'affichage du nombre d'arguments.
 Indication : Utiliser une construction `case` portant sur `$#`.
- Modifier ce script, de façon à ce que, s'il y a un troisième argument, si cet argument est un des caractères `+` `-` `*` `/` `%`, l'opération correspondante soit faite au lieu de l'addition (et si cet argument ne correspond à aucun de ces caractères, un message d'erreur soit affiché). Voir les exemples ci-dessous.
 Indications : Commencer par compléter la construction `case` de la question précédente, et utiliser une construction `case` imbriquée dans la précédente. Il faut inhiber `*`.

Exemples :

```
$ ./calcul 2
Nombre d'arguments insuffisant !
$ ./calcul 103 10
113
$ ./calcul 103 10 \*
1030
$ ./calcul 103 10 /
10
$ ./calcul 103 10 %
3
$ ./calcul 103 10 x
x : Opération non prise en charge
$ ./calcul 103 10 6 +
Trop d'arguments : entrer 2 ou 3 arguments
```

--- * ---

Exercice 5 : `for` sans `in`

- Recopier le script suivant que vous appellerez `arguments`

```
#!/bin/sh
printf "Nombre total d'arguments : %d\n" $#
for chaine; do
    printf '%s\n' "$chaine"
done
```

2. Exécutez ce script des façons suivantes :

```
$ ./arguments a b c
$ ./arguments "a b c"
$ ./arguments
$ ./arguments *
$ ./arguments 367 ab 2
```

3. Ajouter un compteur de façon à afficher `arg. n° <NUM>` avant chaque argument.

4. Pour chaque argument, afficher

- **numérique** si l'argument n'est composé que des chiffres 0, 1, ..., 9
- **non numérique** si l'argument contient un autre caractère que ces chiffres.

--- * ---

Exercice 6 : Script somme

Écrire le script `somme` qui affiche la somme de ses différents arguments. Quelques subtilités (voir exemples de déroulement) :

- Sans argument, 0 est affiché.
- Avec comme premier argument `--help`, un court aide-mémoire est affiché (voir les exemples ci-dessous) et le script se termine. Utiliser la commande `exit` pour mettre fin au script.
- Les arguments non numériques ne comptent pas dans la somme et un avertissement est affiché.

Exemples de déroulement :

```
$ ./somme 5 2 7
14
$ ./somme 10 56
66
$ ./somme 5 aze 7
Attention, argument aze non numérique.
12
$ ./somme
0
$ ./somme --help
Usage: somme [NUM]...
Afficher la somme des nombres entiers donnés en argument.
```

--- * ---

Exercice 7 : Nom de commande et révisions sur la séparation en champs

1. Recopier le script suivant dans un fichier appelé `args` :

```
#!/bin/sh
printf 'Nom de commande : %s\n' "$0"
i=0
for val; do
    i=$(( $i + 1 ))
    printf 'arg. n° %d : %s\n' "$i" "$val"
done
```

2. Pour chacune des commandes suivantes, noter sur feuille l’affichage que vous prévoyez puis vérifier votre réponse en testant la commande (les parenthèses autour d’une commande servent à l’exécuter dans un sous-shell, de façon à ne pas modifier l’état, et en particulier l’IFS du shell courant) :

- a) `$ ./args`
- b) `$ ./././args`
- c) `$ val="a b c d"; ./args $val`
- d) `$ ./args "$val"`
- e) `$ (IFS=; val="a b c d"; ./args $val)`
- f) `$ (IFS='a0'; var='a102a304'; ./args $var)`
- g) `$ (IFS='/'; ./args $HOME)`
- h) `$ (IFS=': '; ./args $PATH)`
- i) `(IFS=''; ./args $(cat args))`

--- * ---

Exercice 8 : La commande `shift` : décaler les paramètres positionnels

Tester les commandes suivantes :

- ```
$ set -- a b c d 'e f g' h
$ printf '$1 vaut %s\n' "$1"
$ shift
$ printf '$1 vaut %s\n' "$1"
$ shift 2
$ printf '$1 vaut %s\n' "$1"
$ printf "nb de param. pos. restants : $#\n"
$ echo "param. pos. restants :"; printf '%s\n' "$@"
```
- On considère la variable chaîne contenant une phrase, par exemple  
`$ chaine="Alice et Bob travaillent sous Unix."`  
 Comment, à l’aide (notamment) d’une séparation en champs, de `set` et `shift` afficher le dernier mot de la chaîne ?

--- \* ---

### Exercice 9 : `set --` : redéfinir les paramètres positionnels

La commande `set` suivie de l’option `--`<sup>1</sup> permet de (re-)définir la valeur (et le nombre) des paramètres positionnels.

- Tester les commandes suivantes :
 

```
$ set -- a b c 'a b c'
$ printf '%d\n' $#
$ printf '$1 vaut %s\n' "$1"
$ printf '$2 vaut %s\n' "$2"
$ printf '$3 vaut %s\n' "$3"
$ printf '$4 vaut %s\n' "$4"
```
- Tester les commandes suivantes (le paramètre spécial `$@` est développé en la suite des paramètres positionnels) :
 

```
$ set -- *
$ printf "nombre de fichiers du rep. courant : $#\n"
$ printf "%s\n" "$@"
$ set -- */
$ printf "nombre de sous-répertoires du rep. courant : $#\n"
$ printf "%s\n" "$@"
```

---

1. En fait l’option `--` met fin aux options.

3. À l'aide de la commande `date`, d'une substitution de commande, et de `set --`, afficher l'heure qu'il est (et seulement cette information) dans le terminal.

--- \* ---

### 3 Exercices d'entraînement sur les scripts

#### Exercice 10 : suffixer

Écrire le script `suffixer` qui prend au moins 2 arguments : le premier est le suffixe et les suivants sont les chaînes auxquelles on veut ajouter le suffixe. Le script produit l'affichage des chaînes avec leur suffixe, ou un message d'erreur si le nombre d'arguments n'est pas au moins deux. Exemples :

```
$./suffixer .c prog hello 'a b'
prog.c
hello.c
a b.c
$./suffixer ' est heureux(se)' Alice Bob
Alice est heureux(se)
Bob est heureux(se)
$./suffixer .c
Usage: suffixer suffixe chaine...
```

--- \* ---

#### Exercice 11 : chgext

Écrire le script `chgext` qui prend en premier argument une nouvelle extension pour des fichiers (par exemple `txt` ou `c` ou `jpg`, ...) et des noms de fichiers existants.

Chacun de ces fichiers est renommé en remplaçant son extension éventuelle par la nouvelle. Un message d'erreur est affiché si le nombre d'arguments est insuffisant.

Exemples :

```
$ ls
a.txt b.truc notes
$ chgext c *
$ ls
a.c b.c notes.c
$ chgext h b.c; ls
a.c b.h notes.c
$ chgext jpeg
Usage: chgext nouv_ext nom_fic...
```

Indication : `IFS=.` et ne pas oublier que la commande `break` permet de sortir d'une boucle.

--- \* ---

#### Exercice 12 : Nom de base

Le *nom de base* d'un nom de chemin est celui que l'on obtient en enlevant le nom d'un éventuel répertoire parent (bref tout ce qui vient avant la dernière oblique).

Par exemple :

- Le nom de base de `/usr/bin` est `bin`.
- Le nom de base de `/home/alice/prog.c` est `prog.c`.
- Le nom de base de `../prog.c` est `prog.c`.

1. Écrire un script `nom_de_base` qui, pour chacun de ses arguments, écrit sur le terminal le nom de base correspondant, c'est-à-dire cet argument privé de son plus long préfixe se terminant par `/`.  
Indication : on pourra se servir de la variable `IFS` ainsi que d'une boucle `for` dont le corps ne contient que l'instruction vide : (deux-points).
2. **Bonus.** Problème : avec le script précédent (et la définition de nom de base que nous avons donnée), le nom de base de `/usr/bin/` (avec une oblique à la fin, comme c'est autorisé pour les noms de répertoire) est la chaîne vide, alors que nous voudrions que ce soit `bin`.  
Autrement dit, il faudrait ignorer une (ou plusieurs) obliques éventuelles à la fin du nom de chemin. Régler ce problème.  
Indications : On peut tester qu'une chaîne est vide en la comparant à l'aide de `case` avec la chaîne vide `"` (ou `'`). On pourra aussi mémoriser la chaîne précédente de la liste dans une autre variable.

--- \* ---